

Parley - Agencia Templates Rewrite

Parley is the functional rewrite of Agencia templates. Every directive is an English phrase wrapped in `{{ ... }}` so it parses left-to-right without backtracking. The grammar is minimal, the runtime remains pure-Go, and every Parley construct evaluates to a new value instead of mutating state.

Sentence Rules

- Keywords are case-insensitive; the guide prints them uppercase so data tokens stand out.
- Names never include spaces or quotes. `planner.tasks` becomes the tokens `planner` and `tasks` .
- Directives describe intent, not control flow. No explicit loops—only declarative listings and maps.
- Results are immutable. Chaining functions returns new values the same way functional languages do.

Core Phrases

Parley phrase	Go template
<code>{{ INPUT }}</code>	<code>{{ .Input }}</code>
<code>{{ INPUT topic }}</code>	<code>{{ .Input "topic" }}</code>
<code>{{ CALL greet_user }}</code>	<code>{{ .Get "greet_user" }}</code>
<code>{{ FACT name FROM greet }}</code>	<code>{{ .Fact "greet.name" }}</code>
<code>{{ IF language FROM greet IS French THEN salutation FROM greet ELSE INPUT ENDIF }}</code>	<code>{{ if eq (.Fact "greet.language") "French" }}{{ .Fact "greet.salutation" }}{{ else }}{{ .Input }}{{ end }}</code>
<code>{{ LIST planner fact }}</code>	<code>{{ range \$item := .Fact "planner.fact" }}- {{ \$item }} {{ end }}</code>
<code>{{ MAP planner fact LIST planner fact AS BULLETS }}</code>	<code>{{ range \$group := .Fact "planner.fact" }}{{ range \$item := \$group }}- {{ \$item }}{{ end }}{{ end }}</code>
<code>{{ COLLECT planner fact LIST planner fact AND FLATTEN }}</code>	<code>{{ range \$item := flatten (collect (.Fact "planner.fact") (tpl "LIST planner fact")) }}- {{ \$item }}{{ end }}</code>

The mapping stays one-to-one: Parley phrases compile to the familiar helpers (`.Input` , `.Fact` , `if` , `range` , and custom collectors). The constrained vocabulary keeps the parser deterministic while preserving Agencia behavior.

Inputs

Use `INPUT` to read the current message or a specific structured input declared on the agent.

```
{{ INPUT }}
{{ INPUT topic }}
```

```
{{ .Input }}
{{ .Input "topic" }}
```

Inputs can feed other expressions, act as arguments to `IF`, or become arguments to higher-order functions. Each directive resolves at render time without mutating chat state.

Facts

Facts are remembered fields stored on the chat, exactly as described in `docs/agencia.md`. The lookup phrase is `FACT <field> FROM <agent>` and compiles to the existing chat fact helper. When the directive already begins with another keyword (`IF`, `LIST`, `MAP`, etc.), you can omit the word `FACT`—a bare name is treated as a fact reference. Only standalone lookups (where the phrase would otherwise start with a fact name) require the explicit `FACT` prefix.

```
{{ FACT greeting FROM concierge }}
```

```
{{ .Fact "concierge.greeting" }}
```

Facts carry their declared types, so the runtime can default them immutably. When a fact might be absent, pair it with an `IF` expression for predictable output.

Conditionals as Expressions

Parley conditionals are expressions that return values. The phrase is `IF <predicate> THEN <value> ELSE <value>`, and both branches must evaluate. Comparators such as `IS`, `IS NOT`, `HAS`, and `EMPTY` map to equality, inequality, membership, and `len(...) == 0`. When `THEN` (or `ELSE`) is left hanging—`THEN }}`—the block that follows becomes the value for that branch, up to the matching `ELSE` or closing `ENDIF`.

```
{{ IF language FROM concierge IS French
    THEN salutation FROM concierge
    ELSE INPUT
ENDIF }}
```

```
{{ if eq (.Fact "concierge.language") "French" }}
  {{ .Fact "concierge.salutation" }}
{{ else }}
  {{ .Input }}
{{ end }}
```

Because the entire directive yields a value, you can choose between complete renderings:

```
{{ IF tone FROM concierge IS Formal
  THEN LIST concierge fact AS SENTENCES
  ELSE LIST concierge fact AS BULLETS
ENDIF }}
```

The helper renders the selected branch while leaving the underlying facts unchanged. Conditionals can also render inline text blocks when you omit the immediate value:

```
{{ IF language FROM concierge IS French THEN }}
Bonjour!
{{ ELSE }}
Howdy!
{{ ENDIF }}
```

```
{{ if eq (.Fact "concierge.language") "French" }}
Bonjour!
{{ else }}
Howdy!
{{ end }}
```

Or mix in previously bound values without a block. Assuming `greeting` and `fallback` were captured earlier with `USING`, you can write:

```
{{ IF language FROM concierge IS French
  THEN THE greeting
  ELSE THE fallback
ENDIF }}
```

```
{{ if eq (.Fact "concierge.language") "French" }}
  {{ $greeting }}
{{ else }}
  {{ $fallback }}
{{ end }}
```

Declarative Listings

`LIST` renders the contents of a fact or input without exposing loops. The first token after `LIST` is the agent (or `INPUT`), the second is the field. Because the directive already begins with `LIST` , you can reference the field directly without repeating `FACT` . Formatting is implied:

- default: whitespace-separated list in square brackets
- `AS BULLETS` : bullet list with `–`
- `AS SENTENCES` : list separated by periods

```
{{ LIST planner fact }}
```

```
{{ LIST planner fact AS BULLETS }}
```

```
{{ $items := .Fact "planner.fact" }}  
[{{ range $index, $item := $items }}{{ if gt $index 0 }} {{ end }}{{ $item }}{{ end  
}}]
```

```
{{ range $item := .Fact "planner.fact" }}  
- {{ $item }}  
{{ end }}
```

The helper returns a formatted string, leaving the original fact untouched.

Higher-Order Composition

Parley treats collection helpers as pure functions. `MAP` , `COLLECT` , and `FLATTEN` accept directives and return new values, allowing pipeline-style composition. The runtime can execute the directives concurrently when the backing fact is marked safe for parallelism.

MAP

`MAP` applies a directive to each element of a source list and concatenates the rendered results. The directive runs with the element bound as the active fact.

```
{{ MAP planner fact  
    LIST planner fact AS BULLETS  
}}
```

```

{{ range $group := .Fact "planner.fact" }}
  {{ range $item := $group }}
    - {{ $item }}
  {{ end }}
{{ end }}

```

Each iteration is independent, so the runtime may schedule them concurrently while preserving order.

COLLECT and FLATTEN

`COLLECT` gathers the outputs of a directive into a list of lists, mirroring functional `mapM`. Add `AND FLATTEN` to drop one layer of nesting, similar to Haskell's `concat`.

```

{{ COLLECT planner fact
  LIST planner fact
}}

```

```

{{ COLLECT planner fact
  LIST planner fact
  AND FLATTEN
}}

```

```

{{ $groups := collect (.Fact "planner.fact") (tpl "LIST planner fact") }}
{{ $flat := flatten $groups }}

```

```

{{ range $group := $groups }}
  [{{ range $index, $item := $group }}{{ if gt $index 0 }} {{ end }}{{ $item }}{{ end
  }}]
{{ end }}

```

```

{{ range $item := $flat }}
  - {{ $item }}
{{ end }}

```

`COLLECT` never mutates the source fact; it returns a fresh slice each time. The optional `AND FLATTEN` shortcut mirrors `flatten (collect ...)` to keep templates concise.

Parley keeps the surface language brief while preserving all Agencia behaviors: structured inputs, persisted facts, pure conditionals, and higher-order collection handling. These sentences provide a functional, immutable layer that compiles directly into the Go templates already shipping in Agencia.

Examples

The curated snippets below highlight every Parley language feature with a side-by-side Go-template translation. Refer back to the sections above for the semantics behind each helper.

- `CALL name` replaces `.Get "name"` .
- `USING label FROM CALL name` captures the result so it can be reused via `THE label` or `USE label` .
- `USING label FROM` (block form) captures the raw text between `USING` and `END` into `THE label` (or `USE label`).
- `CALL name WITH value` passes an explicit argument.
- `LOWER value` and `TRUNCATE value TO n` mirror common Go-template pipelines.

Plain Input

```
Hello, {{ .Input }}!
```

```
Hello, {{ INPUT }}!
```

Named Input

```
Checking out book: {{ .Input "title" }}
```

```
Checking out book: {{ INPUT title }}
```

Agent Invocation

```
{{ .Get "hello_user" }}
```

```
{{ CALL hello_user }}
```

Binding and Reuse

```
{{ $poem := .Get "write_poem" }}
```

```
Poem:  
{{ $poem }}
```

```
Theme:  
{{ .Get "extract_theme" $poem }}
```

```
{{ USING poem FROM CALL write_poem }}
```

Poem:

```
{{ THE poem }}
```

Theme:

```
{{ CALL extract_theme WITH THE poem }}
```

Both `THE poem` and `USE poem` dereference the captured value, so you can choose whichever reads better in context. For example:

```
{{ $summary := .Get "summarize_input" }}
```

Summary:

```
{{ $summary }}
```

```
{{ USING summary FROM CALL summarize_input }}
```

Summary:

```
{{ USE summary }}
```

Binding Raw Text

```
{{ $stmt0 := `This text will  
be bound to the variable 'stmt0'`  
}}
```

```
{{ USING stmt0 FROM }}
```

This text will

be bound to the variable 'stmt0'

```
{{ END }}
```

You can refer back to the captured text using either ``THE stmt0`` or ``USE stmt0``, whichever reads more naturally alongside the surrounding prose.

Facts and Conditionals

```
{{ if eq (.Fact "concierge.language") "French" }}
  {{ .Fact "concierge.salutation" }}
{{ else }}
  {{ .Input }}
{{ end }}
```

```
{{ IF language FROM concierge IS French
  THEN salutation FROM concierge
  ELSE INPUT
  ENDIF }}
```

Listing Collections

```
{{ range $item := .Fact "planner.fact" }}
- {{ $item }}
{{ end }}
```

```
{{ LIST planner fact AS BULLETS }}
```

Mapping Collections

```
{{ range $group := .Fact "planner.fact" }}
  {{ range $item := $group }}
  - {{ $item }}
  {{ end }}
{{ end }}
```

```
{{ MAP planner fact
  LIST planner fact AS BULLETS
}}
```


Collect and Flatten

```
{{ $groups := collect (.Fact "planner.fact") (tpl "LIST planner fact") }}  
{{ $flat := flatten $groups }}
```

```
{{ range $item := $flat }}  
- {{ $item }}  
{{ end }}
```

```
{{ COLLECT planner fact  
    LIST planner fact  
    AND FLATTEN  
}}
```

Pipeline Helpers

```
(Summary: {{ truncate .Input 30 }})
```

```
(Summary: {{ TRUNCATE INPUT TO 30 }})
```

The central theme of the poem is {{ .Input | lower }}.

The central theme of the poem is {{ LOWER INPUT }}.

Other transformations follow the same pattern: replace `.Input` with `INPUT`, substitute `.Get "agent"` with `CALL agent`, and express control flow with the declarative Parley sentences above.